

Ranking Degeneracy: A Learned Wait-Time Model Schedules No Better Than Sorting by Requested Size

Rakshit Rameshbabu
Vellore Institute of Technology, Chennai

September 10, 2026

Abstract

A common recipe in “ML for systems” is to train a regressor that predicts job wait time from cluster-state features and then use its predictions to order the queue. We report that this recipe is *structurally degenerate*. At any single dispatch instant every queued job observes the same cluster, so in the feature vector only the job’s own attributes differ, and in the standard cluster-state feature set every such attribute is a deterministic function of the job’s requested size. The learned score is therefore a function of requested size alone, and the ordering it induces is a per-instant lookup table from size to priority: the cluster-state features shift all scores equally and cannot reorder anything. We verify this over 45,432 real dispatch instants in three settings (a synthetic generator and two Parallel Workloads Archive traces) and find zero counterexamples — two jobs of equal size never receive different scores. The consequences are measurable. On the synthetic benchmark a one-line sort by requested size is statistically *equivalent* to the full XGBoost pipeline (paired TOST $p = 2.6 \times 10^{-16}$, difference CI $[+0.03, +0.23]$ ts against a ± 1.59 ts margin), and an MLP on the same features reproduces the size sort exactly. We then re-ask the whole scheduler comparison on two real traces, replacing the simulated user-estimate model with the estimates the traces actually contain. The synthetic 7.9% mean-wait gain over FCFS does not replicate: the ML scheduler is indistinguishable from FCFS on LANL CM-5 ($p = 0.48$) and any classical policy with a run-time estimate beats it on both machines. We also find that real estimate error is far more damaging to backfill than the over-estimate-only f -model

predicts (+74% mean wait on LANL, where 36% of user estimates are *under*-estimates that the f -model cannot generate). We argue the failure is one of feature design rather than model class, state the condition a wait-time feature set must satisfy to be non-degenerate, and release the harness, traces, and seeds.

1 Introduction

Machine learning for cluster scheduling is an active area, and a recurring design is: (i) log cluster state, (ii) train a regressor to predict a job’s wait time, (iii) at each scheduling decision, score the queued jobs with the regressor and dispatch in ascending predicted wait. The intuition is that a model which has seen how the queue evolves can anticipate contention better than a fixed heuristic.

This paper reports that step (iii) does not do what it appears to do. The argument is elementary but, as far as we can tell, not stated in the literature we build on, and it invalidates the natural reading of any improvement measured this way.

The degeneracy. Fix a dispatch instant t with cluster state S_t and queue Q_t . The feature vector for job j splits into two groups: features of the machine (free processors, queue length, number of running jobs, fragmentation, queue pressure) and features of the job (requested size, and quantities derived from it such as “does it fit now”). The first group is *identical for every job in Q_t* — it describes S_t , not j . So for two jobs $j_1, j_2 \in Q_t$ the predicted waits can differ only through the second group. In the 12-feature set used here (and in the 8-feature

set reconstructible from a Standard Workload Format trace) every member of the second group is a deterministic function of the requested size g_j once S_t is fixed:

$$\begin{aligned}\text{can_fit_now} &= \mathbb{I}[\text{free}(S_t) \geq g_j] \\ \text{gpu_fit_ratio} &= \min(\text{free}(S_t)/g_j, 1) \\ \text{node_availability} &= |\{n : \text{free}_n(S_t) \geq g_j\}|/N \\ \text{queue_pressure} &= (\Sigma_{Q_t} - g_j)/(\text{free}(S_t) + 1).\end{aligned}$$

Hence the score satisfies

$$\hat{w}(j \mid S_t) = g_{S_t}(g_j)$$

for some function g_{S_t} determined entirely by the state, and the induced queue order is the order of g_{S_t} over the sizes present in Q_t . The model is, at each instant, a *lookup table from requested size to priority*. The remaining seven features describe only the cluster — they select *which* table is used — but they cannot distinguish two co-queued jobs, which is the only thing a ranking consumes. (`queue_pressure` does differ between co-queued jobs, but only through the $-g_j$ term: it is itself a function of size given the state, so it cannot break the degeneracy.)

Why this matters. If the argument holds, then a reported improvement of an ML wait-time scheduler over FCFS is evidence about size-based ordering, not about learning. The correct control is not FIFO; it is *sorting by requested size*, which needs no dataset, no training, no inference, no SHAP explanation, and no drift monitor. To our knowledge that control is absent from the ML-scheduling evaluations we are aware of, including our own earlier versions of this work.

Contributions.

1. **The degeneracy, stated and measured** (§4). Over 45,432 dispatch instants across a synthetic generator and two real traces, two equally-sized co-queued jobs never receive different scores (0 violations). Seven of the twelve features vary across the queue in 0.0% of instants, and every feature that does vary is a deterministic function of the requested size. A nine-job queue receives only 2.3–3.1 distinct priority levels.
2. **An equivalence result, not a null result** (§5). Using paired TOST rather than a failure to reject, sorting by requested size is statistically equivalent to the XGBoost scheduler on the synthetic benchmark ($p_{\text{TOST}} = 2.6 \times 10^{-16}$) and on SDSC SP2 ($p_{\text{TOST}} = 1.8 \times 10^{-12}$). An MLP over the same features reproduces the size sort *exactly* on all 20 paired runs.
3. **A trace-driven re-evaluation** (§6). Replaying LANL CM-5 and SDSC SP2 through twelve policies, with the real user runtime estimates the traces contain, the synthetic 7.9% gain over FCFS does not replicate, and every policy with access to a runtime estimate beats the ML scheduler on both machines.
4. **Real estimate error breaks backfill far more than the f -model suggests** (§7). Real estimates are $6.9\times$ the runtime at the median on SDSC but only $1.5\times$ on LANL, where 36% are *under*-estimates — a case the standard over-estimate-only f -model cannot produce. Backfill costs +74% mean wait on LANL under real estimates versus perfect ones, against near-zero sensitivity under the f -model.
5. **A non-degeneracy condition and a reproducible harness** (§8). We state what a feature set must contain to make learning matter for ranking, and release code, traces, and seeds.

2 Related Work

Predicting queue wait time. Wait-time prediction for batch queues is long-standing: Downey’s log-uniform lifetime models, template-based prediction from historical similar jobs, and the QBETS/Karnak line of quantile-based bound predictors all predate modern ML pipelines [1, 5]. The recent line is explicitly ML-based. Park [19] predicts queue wait on a national supercomputer; Lovell et al. [21] use a hierarchical deep model for the same task; Ramachandran et al. [22] combine regressors with metaheuristic hyperparameter search; and Barsanti and Sodan [23]

use predicted wait to drive adaptive resource allocation. Our contribution is not a better predictor. It is an observation about what happens when any such predictor is used as a *ranking* score, and it applies to this whole line: the relevant question for each is whether the feature set contains a per-job attribute that is not a function of requested size given the cluster state (§8).

Backfilling and user runtime estimates.

Backfilling [2] is the dominant production technique. EASY [6] reserves capacity for the queue head and backfills jobs that either finish before the reservation or fit in the surplus beyond the head’s requirement. Conservative backfill reserves for every queued job. Variants refine which job to pick and how much slack to allow: lookahead [12], slack-based priority [14], adaptive reservation [13], probabilistic backfilling on predicted-runtime distributions [11], and deviation backfilling [16]. All depend on user runtime estimates, whose inaccuracy is well documented; the standard model is the f -model, $\hat{r} = r \cdot f$ with $f \sim U(1, C)$ [6], together with the observation that over-estimation can paradoxically *help* backfill [7]. Feitelson [15] shows how sensitive backfilling results are to exactly such evaluation choices. §7 shows the f -model misses a failure mode that real traces exhibit, because it cannot generate an under-estimate at all.

Replacing user estimates with predictions.

The closest prior work replaces the user’s estimate with a system-generated one. Tsafrir et al. [10] predict runtimes from user history and backfill on the prediction; Minh and Wolters [17] predict application runtimes from historical data for the same purpose; Park and Hong [20] improve runtime prediction by conditioning on application type. Yuan et al. [18] note that naively feeding predictions into EASY causes reservation violations and add fairness guarantees to prevent it. This line differs from ours in the quantity predicted: a runtime estimate is a genuine per-job attribute, so ordering by it is *not* degenerate — which is precisely why SJF_USEREST beats the wait-time model in §6, and why a runtime estimate is our first suggested fix.

Learning-based schedulers. Deep-RL approaches [8, 9] learn a policy end-to-end rather than fitting a regressor and sorting by it, and so are not subject to the degeneracy as stated — though the same question (what is the ML-free control?) applies to them. Wang et al. [24] schedule ML cluster jobs from model-level features, an example of a feature set that *does* carry per-job information beyond size. Hovestadt et al. [25] frame the underlying choice as queueing versus planning.

Workloads and metrics. Our synthetic substrate is a simple generator, not a validated workload model; Lublin and Feitelson [26], Downey and Feitelson [27], and Feitelson and Shmueli [28] give the principled alternatives, and §9 treats this as a limitation. Zhang et al. [29] report cross-system variation in job characteristics of the kind that produces our LANL-versus-SDSC divergence. Bounded slowdown [4] and Jain’s index [3] are the standard evaluation metrics; we report both, having found that mean wait alone hides the effect on short jobs. Goponenko et al. [30] argue that these metrics correlate poorly with packing and fairness objectives, which is consistent with the metric-dependent verdicts we obtain in §5.

3 Experimental Setup

3.1 Two evaluation substrates

Synthetic (for continuity). A discrete-time simulator: 8 nodes $\times$ 4 GPUs, 110 jobs arriving uniformly over the first 150 of 300 timesteps, sizes $U\{1, 8\}$, runtimes $U\{5, 20\}$. 20 paired runs (seeds $1000+i$, disjoint from the model’s training seeds $42+i$). This is the substrate all earlier versions of this work used, and we keep it only so the new controls can be compared against the previously published numbers.

Trace-driven (for validity). Two cleaned Parallel Workloads Archive traces: LANL CM-5 (1994, 1024 processors, 122,055 kept jobs) and SDSC SP2 (1998, 128 processors, 43,117 kept jobs). We reserve the first 60% of each trace’s time span for model training and carve 20 evenly spaced evaluation windows from the remainder, each 3 simulated days of warm-up (not measured)

followed by 7 measured days. Mean offered load is 0.72 (SDSC) and 0.68 (LANL). Windows are evenly spaced, *not* selected by load, to avoid cherry-picking congestion.

The trace simulator is event-driven and exact to the second: state changes only at arrivals and completions, so there is no tick-quantisation bias. Because SWF records no per-node placement, we model capacity only; inventing node-level fragmentation would be fabricating data. §9 reports the resulting fidelity gap.

3.2 Policies

Twelve policies. FCFS (first-come first-served with unrestricted first-fit), FIFO_STRICT (a blocked head blocks the queue), SJF_ORACLE (true runtimes; unattainable), SJF_USEREST (the trace’s real requested time), HRRN_USEREST, SMALLEST_FIRST (sort by requested size — the degeneracy control), EASY_USEREST and EASY_ORACLE (canonical two-condition EASY), CONS_BF_USEREST, SRPT_ORACLE (preemptive, with a checkpoint penalty charged as waiting), PROACTIVE (the XGBoost wait-time model, re-trained per trace, no runtime information), and PROACTIVE_EST (the same model retrained with the user estimate as an extra feature — the control for “does ML add anything beyond the estimate?”).

The ML scheduler is given its best case throughout: a model retrained on the same machine’s own earlier data. Zero-shot transfer from the synthetic generator to these traces yields $R^2 \approx 0$ and is not a competitive configuration.

3.3 Metrics and statistics

We report mean wait, p95 wait, **mean bounded slowdown** $\max(\text{turnaround}/\max(r, \tau), 1)$ with $\tau = 60\text{s}$ [4], the Gini coefficient of waits, and utilisation. Wait is defined uniformly as turnaround minus true runtime, so preemptive requeue time and checkpoint overhead are charged to the preemptive policy rather than hidden.

Comparisons are paired (same workload, every policy) with Holm-adjusted t -tests and Wilcoxon signed-rank tests. Crucially, claims that two policies perform *the same* use paired TOST equiva-

lence tests with a margin of 10% of the reference mean; a large p -value from a difference test is not evidence of equivalence, and treating it as such is how a degenerate policy survives review.

4 The Ranking Degeneracy, Measured

We instrumented every PROACTIVE ranking decision in real simulations (45,432 instants with ≥ 2 queued jobs) and recorded, per instant, which features differ across the queue and how the resulting order compares with ML-free orders. Table 1 summarises, and Figure 1 shows the per-feature variation directly.

	Synthetic	SDSC	LANL
Ranking instants	3,646	11,843	29,943
Features (total)	12	8	8
Features varying (mean)	2.62	2.87	2.69
Equal size, diff. score	0	0	0
Distinct scores / queue	2.81	3.09	2.28
Kendall τ vs. size	0.71	0.75	0.62
Order = smallest-first	80.3%	71.5%	77.6%
Order = arrival (FCFS)	19.7%	18.1%	27.5%
Size table monotone	63.1%	57.1%	56.9%

Table 1: The learned score never distinguishes two equally-sized co-queued jobs. Cluster-state features vary across the queue in 0.0% of instants. Source: 05_results/degeneracy/ranking_degeneracy.csv

Three observations. First, the *zero* in row four is the claim: across 45,432 instants there is no counterexample, exactly as the argument in §1 requires. Second, of the twelve synthetic features only five ever differ between co-queued jobs (`job_gpu`, `gpu_fit_ratio`, `can_fit_now`, `node_availability`, `queue_pressure`) and all five are functions of the requested size; the remaining seven vary in 0.0% of instants and are therefore incapable of affecting any ranking. Third, the resolution is very coarse: a queue of ~ 9 jobs is partitioned into only 2.3–3.1 priority classes, and in 18–27% of instants *every* queued job receives the same score, so the policy silently reduces to FCFS.

The recovered size $\rightarrow$ priority table is monotone increasing in only 57–63% of instants. Where it is monotone the policy *is* smallest-job-first; where

A learned wait-time score cannot tell two queued jobs apart by anything but size

Every queued job sees the same cluster, so only per-job features can differ — and each of those is a function of requested size. Across 45,432 dispatch instants (3,646 synthetic, 41,786 replayed from the real SWF traces), two equally-sized jobs received different scores 0 times.

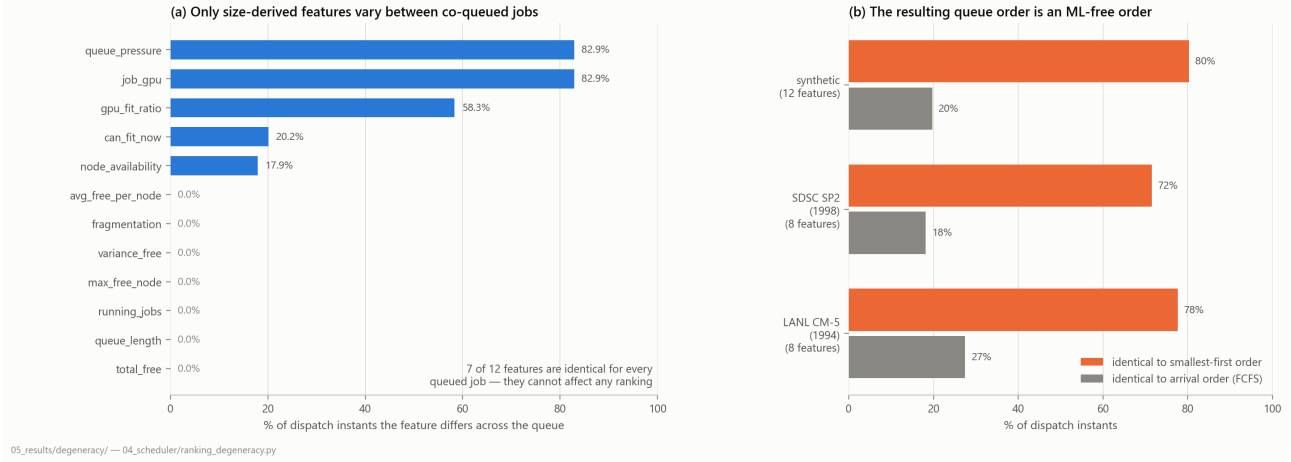


Figure 1: Feature variation in ranking decisions: only job-size features differ across the queue. Of 12 cluster-state features, 7 vary in 0.0% of instants.

it is not, the model has learned a non-trivial size preference. We note explicitly that this fraction does *not* predict where the equivalence of §5 fires — SDSC (57.1%), where the size sort is certified equivalent, and LANL (56.9%), where it is not, are statistically indistinguishable on it. Degeneracy is the claim this section supports; the shape of the learned table is a description of it, not an explanation of the downstream scheduling difference.

5 Sorting by Size Is Equivalent to the Model

Table 2 gives the paired TOST results.

Substrate	Metric	Δ	p_{TOST}	Eq.?
Synthetic	mean wait	+0.80%	2.6×10^{-16}	equiv.
Synthetic	bnd. slow.	+0.69%	3.8×10^{-18}	equiv.
SDSC SP2	mean wait	-0.05%	1.8×10^{-12}	equiv.
SDSC SP2	bnd. slow.	+2.7%	7.4×10^{-4}	equiv.
LANL CM-5	mean wait	+14.4%	0.77	no
LANL CM-5	bnd. slow.	+7.3%	0.30	no

Table 2: SMALLEST_FIRST versus PROACTIVE. Δ is the difference in the metric (positive = the size sort is worse). Equivalence margin 10% of the reference. “Eq.?” is the TOST verdict at that margin. Sources: the two `*equivalence.csv` files under `05_results/`.

On the synthetic benchmark the difference in mean wait is +0.127 ts with a 90% CI of $[+0.025, +0.228]$ against a ± 1.59 ts margin: the entire interval sits well inside the margin, so this is a positive finding of equivalence, not an underpowered null. The MLP baseline is a sharper version of the same point — NN and SMALLEST_FIRST produce *bit-identical* results on every metric of all 20 runs. Two different model classes over the same features collapse to the same one-line sort.

On LANL the equivalence test does not fire, and the honest verdict there is *inconclusive* rather than “different”. The size sort is 14.4% worse on mean wait, but that difference does not survive Holm correction on the paired *t*-test ($p = 0.25$; the Wilcoxon test disagrees at $p = 0.015$), it shrinks to +7.3% and $p = 0.60$ on bounded slowdown, and TOST cannot certify equivalence either ($p = 0.77$). With 20 windows we can neither declare the two policies the same on LANL nor establish that they differ.

We initially attributed this to the learned size table being less monotone on LANL, and that explanation is wrong: the per-instant monotone fraction is 57.1% on SDSC — where equivalence *does* hold — against 56.9% on LANL. Monotonicity does not track where the equivalence fires, so we report the LANL result as underpowered rather than mechanistically explained. What is not in

question on either machine is the degeneracy itself (§4), and on LANL the ML scheduler also fails to beat plain FCFS ($p = 0.48$, §6).

6 Trace-Driven Re-Evaluation

Table 3 reports the twelve policies on both traces, and Figure 2 plots them.

Policy	SDSC SP2		LANL CM-5	
	wait	bsld	wait	bsld
SRPT_ORACLE	60.1	1.1	14.6	1.1
SJF_ORACLE	110.8	13.2	23.0	2.8
SJF_USEREST	115.8	14.7	31.5	4.5
HRRN_USEREST	126.3	16.8	29.5	5.0
PROACTIVE_EST	125.0	14.7	37.0	5.8
SMALLEST_FIRST	145.0	19.0	42.5	6.6
PROACTIVE	145.0	18.5	37.2	6.1
FCFS	174.6	34.7	38.8	8.2
EASY_ORACLE	183.7	35.7	46.0	8.5
EASY_USEREST	195.1	42.5	80.2	20.6
CONS_BF_USEREST	207.5	40.5	113.6	28.4
FIFO_STRICT	755.1	281.5	205.7	71.4

Table 3: Mean wait (minutes) and mean bounded slowdown, averaged over 20 paired 7-day windows per trace at offered load ≈ 0.70 . Source: `trace_schedulers/trace_scheduler_summary.csv`

The synthetic gain does not replicate. On the synthetic benchmark PROACTIVE beats FCFS by 7.4% (and by 7.9% in the older 40-run paired study). On SDSC it beats FCFS by 20.4% ($p = 0.042$, Holm 0.084); on LANL the difference is 4.5% in PROACTIVE’s favour but not distinguishable from zero ($p = 0.48$). A method whose advantage over doing nothing is machine-dependent and insignificant on one of two real machines is not established.

Any runtime signal dominates. SJF_USEREST — shortest-job-first on the estimate the user actually typed — beats PROACTIVE by 20.2% on SDSC (Holm $p = 0.009$) and 15.3% on LANL. This is the same conclusion the synthetic study reached, now on real workloads with real estimate error.

ML adds nothing on top of the estimate. PROACTIVE_EST, which is given the user estimate as a feature and can therefore in principle learn everything SJF exploits *plus* the cluster state, does not beat SJF_USEREST on either machine (+7.9% worse on SDSC, +17.5% on LANL). The ML pipeline’s ceiling here is the sort it is trying to learn.

Tails and fairness. Bounded slowdown separates the policies far more sharply than mean wait, and in the same direction: PROACTIVE sits at 18.5 (SDSC) where SJF_USEREST reaches 14.7 and SRPT_ORACLE 1.1. The reordering policies all buy mean wait with equity (Gini 0.88–0.89 versus 0.83 for FCFS and 0.45 for strict FIFO), reproducing the trade-off the earlier versions of this work reported.

7 Real Estimate Error Versus the f -Model

Because SWF records the user’s requested time, the traces let us measure estimate error instead of simulating it. Table 4 contrasts the two machines with the f -model at $C = 5$ that our synthetic study (and much of the backfill literature) assumes.

	SDSC	LANL	f -model
Estimate present	99.9%	90.8%	—
Median $\hat{r}/r$	6.91	1.51	3.00
Mean $\hat{r}/r$	61.4	23.8	3.00
Within $2\times$	24.0%	41.7%	—
Under-estimates	0.1%	36.3%	0%

Table 4: Real user runtime estimates versus the standard f -model. Source: `trace_schedulers/trace_estimate_quality.csv`

The two machines occupy opposite regimes and neither is the f -model. SDSC is extreme over-estimation (median $6.9\times$, nearly no under-estimates); LANL is roughly calibrated at the median but with a third of jobs under-estimated. The f -model, being $\hat{r} = r \cdot U(1, C)$, cannot produce an under-estimate at all.

This is not a cosmetic difference. Our synthetic study concluded that EASY is “nearly in-

On real workloads the ML scheduler lands where an ML-free size sort lands

Twelve policies replayed through two Parallel Workloads Archive traces, 20 paired 7-day windows each at offered load ≈ 0.70 , using the real user runtime estimates the traces record. Lower is better in both metrics.

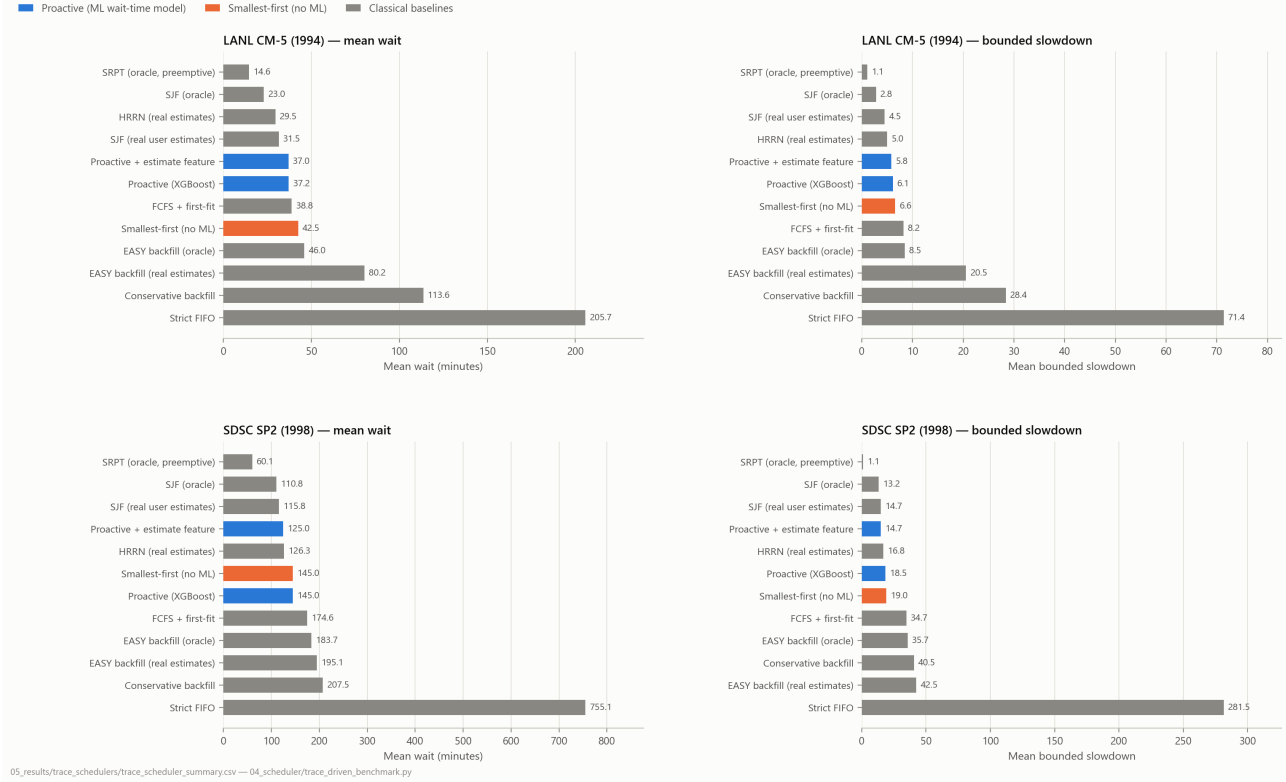


Figure 2: Twelve policies replayed through two real traces. The Proactive (ML) and Smallest-first (ML-free) bars are adjacent; every classical policy with a runtime estimate finishes ahead.

sensitive to estimate quality” (mean wait 19.0–19.8 ts across perfect, f -model, and modal estimates). On the traces, EASY with real estimates costs +6.2% mean wait versus perfect estimates on SDSC — but +74% on LANL (paired t $p = 0.025$ raw but 0.23 after Holm; the Wilcoxon signed-rank test over the same 20 windows gives Holm $p = 2.1 \times 10^{-5}$, so the effect survives correction under the rank test and not under the t -test — the wait distribution’s tail is what carries it). Underestimates break the mechanism EASY depends on: a job that claims it will finish before the reservation and does not *does* delay the head, so the guarantee the reservation is supposed to provide fails exactly when it matters. An evaluation using only over-estimate noise cannot observe this. We therefore think the f -model should not be used as the sole estimate-error model in backfill studies.

8 What Would Make Learning Matter

The degeneracy is a property of the feature set, not of XGBoost. A wait-time model can only produce a non-degenerate ranking if its features distinguish co-queued jobs by something other than requested size. Concretely, the feature vector must contain at least one per-job attribute that is (a) not a function of size given the state, and (b) predictive of wait. Candidates: a runtime estimate (which is precisely what SJF uses, and which §6 shows the model then merely reproduces); job-level history such as the submitting user’s recent runtimes; queue/partition identity; dependency structure; or interaction terms between a per-job attribute and the state (e.g. predicted fit against the future capacity profile rather than the current one).

More broadly, we suggest two evaluation prac-

tices. First, **report the ML-free control that the model’s own feature set implies**: if only the size feature varies across the queue, the control is sorting by size, and FIFO is not an adequate comparison. Second, **use equivalence tests** when arguing that a component matters — had we only run difference tests, the Holm-adjusted $p = 0.17$ comparison between the size sort and the XGBoost pipeline would have been reported as “no significant difference” and quietly dropped, rather than as the finding it is.

9 Threats to Validity

Simulator fidelity. The capacity-only model omits node-level placement and site policy (partitions, per-user limits, administrative holds), which SWF does not record. Against the recorded waits our simulated FCFS is close on LANL (38.8 vs. 33.2 min mean; the simulator is slightly *pes-simistic*) but far more permissive on SDSC (174.6 vs. 630.9 min mean, against a recorded *median* of 19.3 min). SDSC’s recorded mean is dominated by an extreme tail our model cannot reproduce. SDSC results should therefore be read as “a correctly-loaded 128-processor machine driven by the real arrival, size, runtime and estimate distributions”, not as a replay of SDSC’s actual queue. The degeneracy result does not depend on this: it is a property of the feature vector, and holds in all three substrates.

Statistical power. Twenty windows per trace gives limited power; after Holm correction across eleven comparisons many individually plausible differences are not significant, particularly on LANL. We therefore lean on effect sizes, on the consistency of the ordering across both machines, and — for the sameness claims — on equivalence tests rather than on failures to reject.

Covariate shift in the retrained model. The per-trace model is trained on features reconstructed from the *recorded* schedule, produced by the site’s real scheduler. When PROACTIVE runs it induces different states than those it was trained on. This is inherent to the approach rather than an artefact of our setup, but it means the model

is evaluated slightly off-distribution even after re-training.

Vintage of the traces. LANL CM-5 (1994) and SDSC SP2 (1998) are batch HPC machines. Their users supply runtime estimates, which is exactly the regime in which the ML scheduler has no niche. Modern deep-learning clusters, where job durations are often genuinely unknown at submit time, are the setting in which a zero-runtime-information policy could still be worth something; validating there is the clearest next step, and we make no claim about it here.

10 Conclusion

We set out to find where an ML wait-time scheduler adds value and found that, as constructed, it cannot add value to queue ordering: its score is a function of requested size alone, verified over 45,432 dispatch instants without a single counterexample. A one-line sort is statistically equivalent to it on two of three substrates, an MLP over the same features reproduces the sort exactly, its synthetic advantage over FCFS does not replicate on real traces, and any classical policy holding a runtime estimate beats it on both real machines. Alongside this we report that real user-estimate error damages backfill far more than the standard f -model predicts, because a third of LANL’s estimates are under-estimates that the f -model cannot generate.

We report these as negative results because they are the results. The constructive content is the non-degeneracy condition of §8 and the methodological point that an ML systems component should be compared against the ML-free control its own feature set implies — with an equivalence test, not a failed difference test.

Artifact Availability

Code, both SWF traces, the seeded pipeline (`run_all_experiments.sh`), and every CSV behind the tables are in the repository. `04_scheduler/ranking_degeneracy.py` reproduces §4, `trace_driven_benchmark.py` repro-

duces §6–7, and `multi_scheduler_benchmark.py` reproduces §5.

References

- [1] D. G. Feitelson, *Workload Modeling for Job Scheduling in Parallel Computers*, EWOPP 1997.
- [2] A. Yoo et al., *SLURM: Simple Linux Utility for Resource Management*, JSSPP 2003.
- [3] R. Jain, D.-M. Chiu, and W. Hawe, *A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems*, DEC Research Report TR-301, 1984.
- [4] D. G. Feitelson, *Metrics for Parallel Job Scheduling and Their Convergence*, JSSPP 2001.
- [5] D. G. Feitelson, D. Tsafir, and D. Krakov, *Experience with Using the Parallel Workloads Archive*, J. Parallel Distrib. Comput. 74(10):2967–2982, 2014. <https://www.cs.huji.ac.il/labs/parallel/workload/>
- [6] A. W. Mu’alem and D. G. Feitelson, *Utilization, Predictability, Workloads, and User Runtime Estimates in Scheduling the IBM SP2 with Backfilling*, IEEE TPDS 12(6), 2001.
- [7] D. Tsafir, Y. Etsion, and D. G. Feitelson, *Modeling User Runtime Estimates*, JSSPP 2005.
- [8] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, *Resource Management with Deep Reinforcement Learning*, HotNets 2016.
- [9] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, *Learning Scheduling Algorithms for Data Processing Clusters*, SIGCOMM 2019.
- [10] D. Tsafir, Y. Etsion, and D. G. Feitelson, *Backfilling Using System-Generated Predictions Rather Than User Runtime Estimates*, IEEE TPDS 18(6):789–803, 2007.
- [11] A. Nissimov and D. G. Feitelson, *Probabilistic Backfilling*, JSSPP 2007.
- [12] E. Shmueli and D. G. Feitelson, *Backfilling with Lookahead to Optimize the Packing of Parallel Jobs*, JSSPP 2003.
- [13] E. Shmueli and D. G. Feitelson, *Backfilling with System-Generated Job Runtime Estimates*, IPDPS 2005.
- [14] D. Talby and D. G. Feitelson, *Supporting Priorities and Improving Utilization of the IBM SP Scheduler Using Slack-Based Backfilling*, IPPS/SPDP 1999.
- [15] D. G. Feitelson, *Experimental Analysis of the Root Causes of Performance Evaluation Results: A Backfilling Case Study*, IEEE TPDS 16(2):175–182, 2005.
- [16] T. H. L. Hai, K. N. Duy, T. N. Manh, D. M. Hoang, and N. Thoai, *Deviation Backfilling: A Robust Backfilling Scheme for Improving the Efficiency of Job Scheduling on High Performance Computing Systems*, IEEE, 2023.
- [17] T. N. Minh and L. Wolters, *Using Historical Data to Predict Application Runtimes on Backfilling Parallel Systems*, IEEE, 2010.
- [18] Y. Yuan, Y. Wu, W. Zheng, and K. Li, *Guarantee Strict Fairness and Utilize Prediction Better in Parallel Job Scheduling*, IEEE TPDS, 2014.
- [19] J.-W. Park, *Queue Waiting Time Prediction for Large-scale High-performance Computing System*, IEEE, 2019.
- [20] J.-W. Park and T. Hong, *Improving Runtime Prediction Performance Based on Application Type of Jobs on Large-scale Cluster System*, IEEE, 2025.
- [21] A. Lovell, P. Wisniewski, S. Rodenbeck, et al., *A Hierarchical Deep Learning Approach for Predicting Job Queue Times in HPC Systems*, IEEE, 2024.
- [22] S. Ramachandran, M. L. Jayalal, et al., *Combining Machine Learning and Metaheuristic Algorithms for Predicting Waiting Time of*

- High Performance Computing Jobs*, IEEE, 2024.
- [23] L. Barsanti and A. C. Sodan, *Adaptive Job Scheduling via Predictive Job Resource Allocation*, JSSPP 2006.
 - [24] H. Wang, Z. Liu, and H. Shen, *Machine Learning Feature Based Job Scheduling for Distributed Machine Learning Clusters*, IEEE/ACM Trans. Networking 31(1):58–72, 2023.
 - [25] M. Hovestadt, O. Kao, A. Keller, and A. Streit, *Scheduling in HPC Resource Management Systems: Queuing vs. Planning*, JSSPP 2003.
 - [26] U. Lublin and D. G. Feitelson, *The Workload on Parallel Supercomputers: Modeling the Characteristics of Rigid Jobs*, J. Parallel Distrib. Comput. 63(11):1105–1122, 2003.
 - [27] A. B. Downey and D. G. Feitelson, *The Elusive Goal of Workload Characterization*, ACM SIGMETRICS Perf. Eval. Rev. 26(4):14–29, 1999.
 - [28] D. G. Feitelson and E. Shmueli, *A Case for Conservative Workload Modeling: Parallel Job Scheduling with Daily Cycles of Activity*, MASCOTS 2009.
 - [29] D. Zhang, M. S. Raj, B. Xie, S. Di, and D. Dai, *Cross-System Analysis of Job Characterization and Scheduling in Large-Scale Computing Clusters*, IEEE, 2024.
 - [30] A. V. Goponenko, K. Lamar, C. Peterson, B. A. Allan, J. M. Brandt, and D. Dechev, *Metrics for Packing Efficiency and Fairness of HPC Cluster Batch Job Scheduling*, IEEE, 2022.